

Автономная навигация и построение карты в цифровом двойнике

ROS2 Humble + Nav2 + SLAM Toolbox

Цель работы: Изучение стека автономной навигации Nav2 и алгоритмов одновременной локализации и картографирования (SLAM) применительно к симуляционной среде цифрового двойника. Настройка costmap, планировщика пути и контроллера движения. Построение карты виртуального помещения и сравнительный анализ с картой реального робота из лабораторной работы №1 однопользователя.

Ход работы

1.1. Стек Nav2 и его компоненты

Nav2 (Navigation2) — официальный навигационный стек ROS2. В отличие от ROS1 где навигация была монолитным пакетом, Nav2 построен как набор независимых серверов действий (action servers), каждый из которых решает одну задачу.

BT Navigator — верхний уровень. Выполняет дерево поведений (Behavior Tree), которое описывает общую логику навигации: планируй путь → следуй по нему → если застрял — восстановись. Дерево задаётся XML-файлом и легко модифицируется без перекомпиляции.

Planner Server — глобальный планировщик. Получает целевую точку и строит оптимальный путь по глобальной costmap от текущей позиции до цели. По умолчанию используется алгоритм NavFn (вариант Дейкстры/A*).

Controller Server — локальный контроллер. Получает глобальный путь и генерирует команды скорости (/cmd_vel), объезжая динамические препятствия которых не было на карте. По умолчанию используется DWB (Dynamic Window Approach).

Costmap 2D — двумерная карта стоимостей. Каждая ячейка содержит значение от 0 (свободно) до 254 (летально опасно). Существует в двух экземплярах: глобальная (вся карта, обновляется редко) и локальная (окрестность робота, обновляется на каждом скане лидара).

Recovery Behaviors — действия при зависании: развернуться на месте, отъехать назад, очистить costmap.

1.2. SLAM Toolbox

SLAM Toolbox — современная реализация SLAM для ROS2, пришедшая на смену Gmapping. Ключевые отличия от Gmapping:

Gmapping использует фильтр частиц (Particle Filter) — каждая частица это отдельная гипотеза карты. При большом числе частиц точность высокая, но потребление памяти растёт линейно. SLAM Toolbox использует граф поз (Pose Graph) — робот оставляет узлы-снимки сканов, а алгоритм оптимизирует связи между ними. Это позволяет строить карты больших помещений с меньшими ресурсами.

Loop Closure (замыкание петли) — ключевой механизм коррекции ошибок. Когда робот возвращается в ранее посещённое место, SLAM сравнивает текущий скан с сохранёнными и корректирует весь граф поз разом. Без замыкания петли ошибки одометрии накапливаются и карта «расползается».

SLAM Toolbox работает в двух режимах: mapping (построение новой карты) и localization (локализация на готовой карте).

1.3. Costmap и inflation radius

Inflation radius — один из важнейших параметров навигации. Вокруг каждого препятствия costmap создаёт зону повышенной стоимости радиусом inflation_radius. Смысл: робот не точка, у него есть корпус. Если планировщик не знает о размере робота, он проложит путь вплотную к стене и робот заденет её.

robot_radius задаёт размер робота. Для TurtleBot3 Burger это 0.105 м. Inflation radius должен быть не меньше robot_radius, обычно берут 1.5–2× от него.

Слишком большой inflation radius делает узкие проходы непроходимыми — планировщик считает их заблокированными. Слишком маленький — робот задевает стены. Подбор этого параметра под конкретное помещение является одним из спецзаданий данной работы.

1.4. Action-интерфейс Nav2

Навигация запускается через ROS2 Actions — механизм для долгих асинхронных задач с обратной связью. В отличие от сервисов (запрос-ответ), action позволяет получать промежуточные статусы выполнения и отменять задачу.

Цель навигации отправляется на action server navigate_to_pose. Запрос содержит целевую позу (позицию + ориентацию) в системе координат карты. В процессе выполнения сервер публикует feedback с текущим расстоянием до цели.

Из командной строки:

```
bash
```

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 0.5, z: 0.0},
  orientation: {w: 1.0}}}}"
```

1.5. Сравнение карт симуляции и реального робота

Карта сохраняется в двух файлах. .pgm — изображение в градациях серого: белый (254) = свободно, чёрный (0) = занято, серый (205) = неизвестно. .yaml — метаданные: разрешение (метров на пиксель), координаты origin (левый нижний угол карты), порог occupied_thresh.

Для сравнения карт важно понимать что карта симуляции будет геометрически точной (виртуальные стены идеальны), а карта реального робота будет содержать шум от сенсора, артефакты от динамических объектов (люди, стулья) и ошибки одометрии. Качественное сравнение включает: совпадение общего контура помещения, наличие/отсутствие артефактов, чёткость границ стен.

2.1. Установка Nav2 и SLAM Toolbox

bash

```
sudo apt install -y \  
ros-humble-navigation2 \  
ros-humble-nav2-bringup \  
ros-humble-slam-toolbox
```

Проверить установку:

bash

```
ros2 pkg list | grep nav2  
ros2 pkg list | grep slam
```

2.2. Запуск симуляции с роботом

Убедиться что пакет собран и робот присутствует в сцене (согласно лабораторной работе №2). Запустить симуляцию:

bash

```
source ~/ros2_ws/install/setup.bash  
export TURTLEBOT3_MODEL=burger  
ros2 launch lab_digital_twin simulation.launch.py
```

В отдельном терминале убедиться что данные лидара поступают:

bash

```
ros2 topic hz /scan  
# Ожидаем: average rate: 10.000
```

2.3. Запуск SLAM

bash

```
ros2 launch slam_toolbox online_async_launch.py \  

```

```
slam_params_file:=/opt/ros/humble/share/slam_toolbox/config/mapper_params_online_async.yaml \
```

```
use_sim_time:=true
```

Флаг `use_sim_time:=true` критически важен — без него SLAM будет использовать системное время, а не время симуляции Gazebo, и карта будет строиться некорректно.

Проверить что карта публикуется:

```
bash
```

```
ros2 topic echo /map --once
```

Ожидаем: заголовок с `frame_id: 'map'`, массив `data`

2.4. Визуализация в RViz2

```
bash
```

```
ros2 run rviz2 rviz2
```

Добавить следующие отображения:

- Map → топик `/map` — строящаяся карта
- LaserScan → топик `/scan` — данные лидара
- RobotModel → топик `/robot_description` — модель робота
- TF — дерево трансформаций (убедиться что видны фреймы `map`, `odom`, `base_link`)

Установить Fixed Frame: `map`.

2.5. Построение карты — ручное управление

Запустить телеуправление:

```
bash
```

```
ros2 run turtlebot3_teleop teleop_keyboard
```

Методика объезда для качественной карты:

1. Двигаться вдоль стен медленно (линейная скорость не более 0.15 м/с).
2. На поворотах останавливаться на 2–3 секунды — SLAM стабилизирует позицию.
3. Объехать всё помещение и вернуться в точку старта — это запустит механизм Loop Closure. В RViz2 будет видно как карта резко выравнивается.
4. Избегать симметричных коридоров без уникальных ориентиров — SLAM может перепутать похожие участки.

В процессе объезда наблюдать в RViz2 как серые (неизвестные) зоны заполняются белым (свободно) и чёрным (стены).

2.6. Сохранение карты

bash

```
mkdir -p ~/ros_labs/lab3
```

```
ros2 run nav2_map_server map_saver_cli \  
-f ~/ros_labs/lab3/sim_map
```

Команда создаёт два файла: `sim_map.pgm` и `sim_map.yaml`. Изучить содержимое `yaml`:

bash

```
cat ~/ros_labs/lab3/sim_map.yaml
```

Обратить внимание на параметр `origin` — это координаты левого нижнего угла карты относительно точки старта робота, и `resolution` — размер одного пикселя в метрах.

2.7. Запуск автономной навигации по готовой карте

После сохранения карты остановить SLAM и запустить Nav2 в режиме локализации:

bash

Терминал 1 — локализация на готовой карте

```
ros2 launch nav2_bringup localization_launch.py \  
map:=~/ros_labs/lab3/sim_map.yaml \  
use_sim_time:=true
```

Терминал 2 — навигационный стек

```
ros2 launch nav2_bringup navigation_launch.py \  
use_sim_time:=true
```

В RViz2 нажать кнопку **2D Pose Estimate** и указать начальную позицию робота на карте — это необходимо чтобы AMCL (локализация по готовой карте) понял где робот находится.

Затем нажать **Nav2 Goal** и кликнуть целевую точку на карте. Робот должен самостоятельно построить путь и поехать к цели.

2.8. Отправка цели из командной строки

bash

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
    "{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 0.5, z: 0.0},
orientation: {w: 1.0}}}}"
```

Наблюдать feedback в консоли — текущее расстояние до цели должно уменьшаться.

2.9. Сравнение с картой реального робота

Получить от одноклассника файл `real_map.pgm` из его лабораторной работы №1. Открыть оба файла в любом просмотрщике изображений и провести качественный анализ по следующим критериям:

Критерий	Карта симуляции	Карта реального робота
Чёткость границ стен	Идеальные прямые линии	Размытые, с шумом сенсора
Артефакты	Отсутствуют	Ножки столов, люди, блики
Неизвестные зоны	Только необъезженные участки	Могут возникать от отражений
Точность контура	Совпадает с реальным планом	Зависит от качества объезда